Week 5 – Implementation Progress Report

Team P3

Cross-Platform Budgeting Application Project

Karunakar Reddy Kaitha, Jordan Lee and John Lim

Computer Science and Information Systems Department, Regis University

MSSE 692: Software Engineering Practicum I

Morgan Worrell

February 15, 2026

1. Executive Summary

Week 5 focused on transitioning from architectural design to functional implementation. The team completed the majority of backend MVP functionality and initiated frontend GUI development. Core authentication, budgeting, income, expense, and reporting features are now operational and persisted in PostgreSQL. Unit tests are implemented for all major service-layer components, and the API contract is documented and automatically published via GitHub Pages.

While backend feature development is largely complete at the MVP level, integration testing and frontend completion remain in progress. CI is functional but not yet fully hardened (e.g., coverage reporting and branch protection enforcement). Core financial views are still under development.

Overall, we are on track for Week 6 goals, but frontend integration complexity and integration testing represent the primary risks.

2. What Was Completed This Week

Backend Implementation

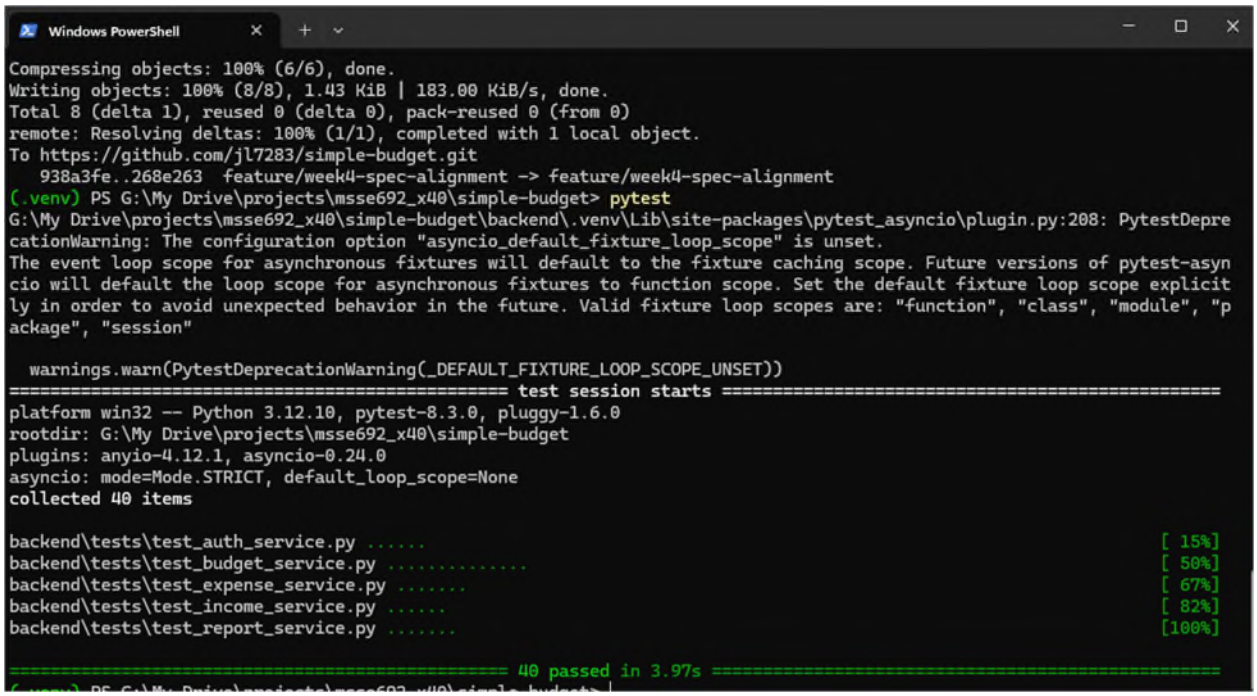
The following backend components were completed:

- User registration and JWT login
- Protected endpoints with authentication middleware
- Monthly budget creation and update
- Income tracking endpoints
- Expense tracking with category grouping
- Monthly summary reporting with aggregation logic
- Database schema validated with PostgreSQL
- Strict month-boundary logic validation in report service

All endpoints are functional and tested locally.

Testing Implementation

- 40+ unit tests implemented
- Service-layer tests use mocking for deterministic security validation
- Error code and envelope consistency validated
- Month boundary logic explicitly tested
- All tests passing locally

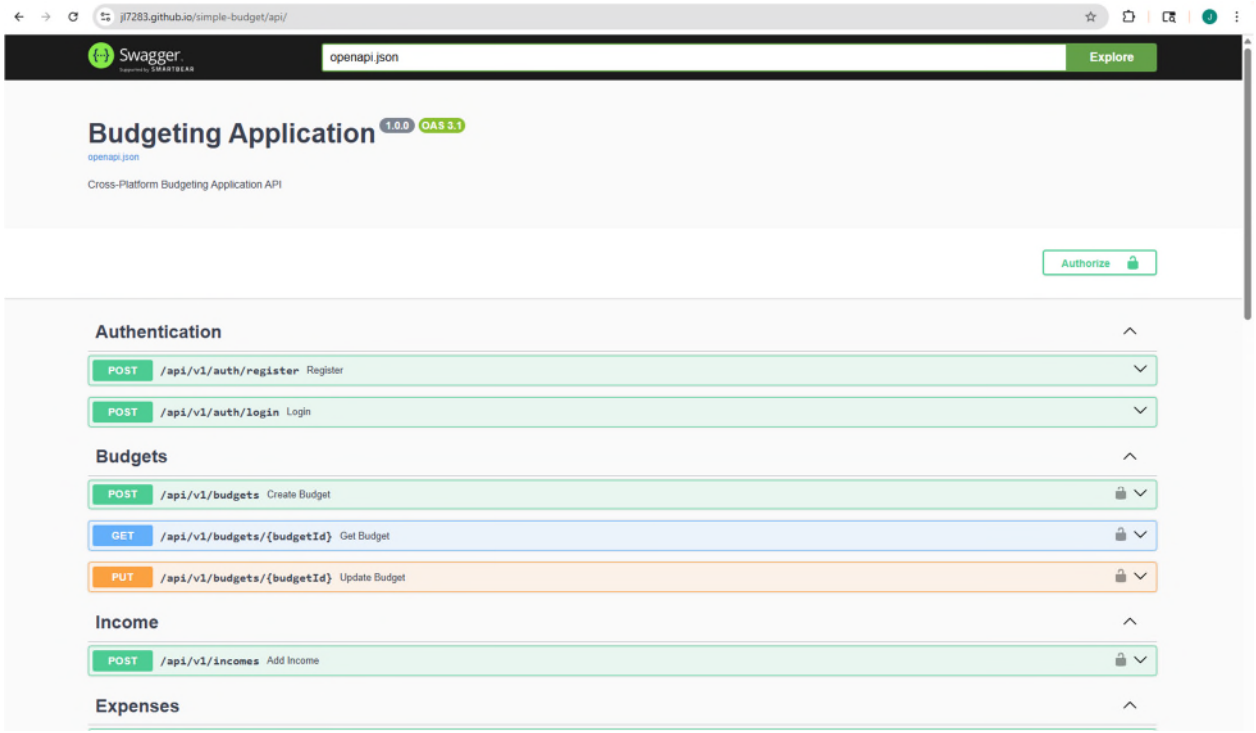


```
Windows PowerShell
Compressing objects: 100% (6/6), done.
Writing objects: 100% (8/8), 1.43 KiB | 183.00 KiB/s, done.
Total 8 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/jl7283/simple-budget.git
   938a3fe..268e263  feature/week4-spec-alignment -> feature/week4-spec-alignment
(.venv) PS G:\My Drive\projects\msse692_x40\simple-budget> pytest
G:\My Drive\projects\msse692_x40\simple-budget\backend\.venv\Lib\site-packages\pytest_asyncio\plugin.py:208: PytestDepre
cationWarning: The configuration option "asyncio_default_fixture_loop_scope" is unset.
The event loop scope for asynchronous fixtures will default to the fixture caching scope. Future versions of pytest-asyn
cio will default the loop scope for asynchronous fixtures to function scope. Set the default fixture loop scope explicit
ly in order to avoid unexpected behavior in the future. Valid fixture loop scopes are: "function", "class", "module", "p
ackage", "session"

  warnings.warn(PytestDeprecationWarning(_DEFAULT_FIXTURE_LOOP_SCOPE_UNSET))
============================================= test session starts =============================================
platform win32 -- Python 3.12.10, pytest-8.3.0, pluggy-1.6.0
rootdir: G:\My Drive\projects\msse692_x40\simple-budget
plugins: anyio-4.12.1, asyncio-0.24.0
asyncio: mode=Mode.STRICT, default_loop_scope=None
collected 40 items

backend\tests\test_auth_service.py ......                                                          [ 15%]
backend\tests\test_budget_service.py ..............                                                [ 50%]
backend\tests\test_expense_service.py .......                                                      [ 67%]
backend\tests\test_income_service.py ......                                                        [ 82%]
backend\tests\test_report_service.py .......                                                       [100%]

============================================= 40 passed in 3.97s ==============================================
```

Screenshot 1: Pytest successful test run

API Documentation

- OpenAPI specification exported automatically
- Swagger UI generated and published via GitHub Actions
- Documentation publicly accessible



Screenshot 2: Hosted Swagger UI page

Documentation URL:

https://jl7283.github.io/simple-budget/api/

GitHub Repo URL:

https://github.com/jl7283/simple-budget

GUI Progress

- Frontend development has begun.
- Budget, income, and reporting views are under development.

3. What's Working vs What's Not

Working

Backend:

- All core REST endpoints respond correctly
- Authentication flow validated
- Data persists correctly in PostgreSQL
- Report aggregation produces correct totals
- Error responses conform to standardized envelope

Infrastructure:

- Local environment setup stable across team
- CI runs automated tests
- OpenAPI auto-publication functional

Not Working / Incomplete

Backend:

- No automated integration tests yet
- No load testing performed
- No performance benchmarking
- Logging minimal (not production-grade)

Frontend:

- Budget creation UI not fully implemented
- Expense entry UI incomplete
- Report visualization not yet implemented
- Error display handling basic

CI/CD:

- Branch protection not yet enforced
- Code coverage not yet measured
- No automated deployment to hosting environment

4. Infrastructure & Development Environment

Version Control & Branching

- GitHub repository active
- Feature branches used for development
- Pull requests used before merge
- Informal code review conducted

Branch protection enforcement planned for Week 6.

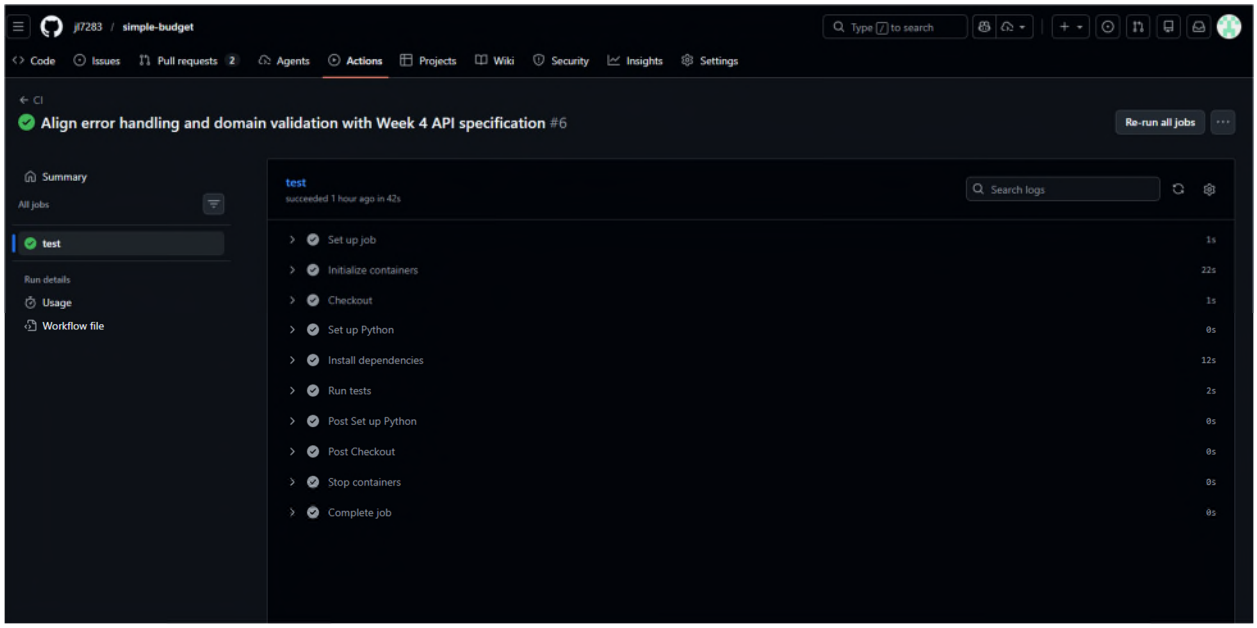
CI/CD Status

GitHub Actions currently performs:

- Automated pytest execution on push
- OpenAPI export
- Documentation publication to GitHub Pages

Not yet implemented:

- Code coverage reporting
- Deployment automation
- Required status checks before merge



Screenshot 3: GitHub Actions successful workflow run

Development Environment

All team members:

- Successfully run backend locally
- Successfully connect to PostgreSQL
- Successfully run full test suite

No active environment blockers.

Build & Dependency Management

Backend:

- Virtual environment isolation
- requirements.txt managed dependencies
- Environment variable configuration via .env

Frontend:

- Node/npm based dependency management
- Local dev server functioning

5. Implementation Status by Component

Component	Status	Completion	Owner	Blockers
Authentication	Complete	100%	Backend	None
Budget API	Complete	100%	Backend	None
Income API	Complete	100%	Backend	None
Expense API	Complete	100%	Backend	None
Reporting API	Complete (MVP)	95%	Backend	Integration validation pending
Unit Testing	Complete (Service Layer)	90%	Backend	No coverage metrics
Integration Testing	In Progress	30%	Backend	Time allocation
CI Pipeline	Functional	70%	DevOps	Coverage + enforcement
GUI Budget View	In Progress	40%	Frontend	API integration
GUI Expense View	In Progress	30%	Frontend	Form validation
GUI Reporting View	Not Started	0%	Frontend	Dependent on UI components

Backend is feature complete for MVP, but integration maturity is still improving.

6. Testing & Quality

Unit Test Coverage

- Approximately 40+ unit tests
- Service-layer logic heavily tested
- Security functions mocked for determinism
- Boundary cases tested (e.g., month formatting)
- All tests passing

Formal coverage measurement tool not yet implemented. Estimated service-layer coverage exceeds 80%, but controller and repository layers have limited direct coverage.

Integration Testing

Manual integration tests performed using:

- Swagger UI
- curl

Automated integration testing not yet implemented.

Code Review Process

- Pull requests reviewed before merge
- Informal peer review
- CI validates build

Branch protection enforcement pending.

Known Issues

- Limited integration automation
- No performance metrics
- GUI incomplete
- Minimal logging

No critical functional bugs identified.

7. Velocity & Week 6 Planning

Velocity Assessment

Planned Week 5 Goals:

- Complete backend MVP
- Implement unit testing
- Initiate GUI
- Establish CI

Achieved:

- Backend MVP complete
- Unit testing implemented
- CI configured

Completion Rate: ~80% of planned deliverables

Carryover:

- Integration testing
- GUI financial views
- CI enforcement enhancements

Lessons Learned

- Integration between authentication middleware and frontend required additional debugging.
- Report aggregation boundary conditions required careful validation.
- CI module path configuration caused unexpected issues.
- Frontend state management requires more planning than initially estimated.

8. Week 6 Plan

Week 6 objectives:

1. Complete GUI views for:
    - Budget creation
    - Expense entry
    - Monthly summary display

2. Implement automated integration tests

3. Add code coverage reporting to CI

4. Enforce branch protection

5. Conduct regression testing

6. Stabilize frontend-backend integration

Primary risk:

Frontend complexity and ensuring API contract stability during integration.

9. Scope Adjustments

No core functional features removed.

Deferred items:

- Performance optimization
- Advanced analytics
- Production monitoring tooling
- Load testing

Focus remains on delivering a stable full-stack MVP.

10. Overall Assessment

The project has transitioned successfully from architecture to functional implementation. Backend MVP functionality is complete and stable. GUI development is underway with authentication integrated. Testing is strong at the unit level but requires integration automation. CI is operational but will be strengthened next week.

The team is on track for Week 6 goals, with frontend integration representing the primary remaining challenge.